

04 — Linked List

DSA in Python — Interview Prep

Linked List — Tutorial

Covers the 10 problems solved in this folder, plus the shared `list_node.py` (a `ListNode` class and `to_linked_list/ to_python_list` conversion helpers used by every test harness here).

Linked lists have no random access (`nums[i]` doesn't exist - you can only follow `.next`), so every technique in this folder exists to work around that one constraint using a small number of pointers moving through the list at different speeds or in different phases.

The patterns

1. In-place reversal (01)

Walk once, flip each node's `.next` to point backward. The one non-obvious detail: save `next_node = curr.next` *before* overwriting `curr.next = prev`, otherwise the rest of the list is lost the moment you rewrite the pointer. This exact three-line reversal core reappears inside 07 and 08.

2. Dummy head for merge/splice (02, 06)

Whenever a problem might need to change *which node is first* (merging, deleting the head), give yourself a throwaway `dummy` node pointing at the real head, operate relative to `dummy`, and return `dummy.next` at the end. This removes the need for an `if this is the first node` special case anywhere in the main logic.

3. Fast-slow pointers (03, 04, 05)

The core linked-list technique, first previewed in

`03_two_pointers_sliding_window/10_find_the_duplicate_number` on a plain array. `fast` moves 2 steps per iteration, `slow` moves 1:

- **Cycle detection (03)**: if a cycle exists, `fast` eventually laps `slow` inside it and they land on the same node. If acyclic, `fast` simply reaches `None` first.
- **Finding the cycle's start (04)**: two-phase version of the same idea - phase 1 finds *a* meeting point, phase 2 (reset one pointer to `head`, advance both by 1 step) finds the meeting point is always exactly the cycle's entrance. Worth being able to state the informal proof (see the file's docstring), not just the code.
- **Finding the middle (05)**: no meeting point needed here - by the time `fast` runs off the end, `slow` has covered exactly half the distance, landing on the middle directly.

4. Two pointers with a fixed gap (06)

`remove_nth_node_from_end` advances one pointer `n` steps first to create a gap, then moves both together until the leading pointer falls off the end. The trailing pointer ends up exactly `n` nodes from the end without ever computing the list's length - a one-pass alternative to the two-pass "count length, then walk to position" approach.

5. Composite problems — combining earlier patterns (07, 08)

Once reversal and middle-finding both exist as primitives, harder problems become "apply pattern A, then pattern B, then a simple walk":

- `palindrome_linked_list`: find middle (05) -> reverse second half (01) -> walk both halves comparing values.
- `reorder_list`: find middle (05) -> reverse second half (01) -> merge by strict alternation (structurally like 02's merge, but alternating instead of comparison-driven, since the two halves aren't sorted relative to each other).

This is the main payoff of solving problems in a deliberate order: by the time you reach a "hard"-rated problem, it's often just 2-3 "easy" problems chained together.

6. Two-pointer list-switching for alignment (09)

`intersection_of_two_linked_lists` uses a pointer-switching trick instead of computing list lengths up front: each pointer walks its own list, then switches to the *other* list's head the instant it hits the end. Both pointers then cover exactly $\text{len}(A) + \text{len}(B)$ total nodes by the time they'd meet, which automatically cancels out any length difference between the two lists - no separate "compute the offset" step needed.

7. Doubly linked list + hashmap for O(1) design (10)

`lru_cache` is the one problem here needing a doubly (not singly) linked list. The dict gives O(1) key -> node lookup; the doubly linked list keeps nodes ordered by recency and supports O(1) removal of an arbitrary node (not just the head/tail), because removal only needs to patch two neighbor pointers - no need to walk from the head to find a predecessor, which is exactly the operation a singly linked list can't do in O(1). Sentinel `head/tail` nodes (not real entries) avoid null-checking to know when a queue of one is "empty".

Complexity summary

#	Problem	Time	Extra space
01	<code>reverse_linked_list</code>	O(n)	O(1)
02	<code>merge_two_sorted_lists</code>	O(n+m)	O(1)
03	<code>linked_list_cycle</code>	O(n)	O(1)
04	<code>linked_list_cycle_ii</code>	O(n)	O(1)
05	<code>middle_of_linked_list</code>	O(n)	O(1)
06	<code>remove_nth_node_from_end</code>	O(n)	O(1)
07	<code>palindrome_linked_list</code>	O(n)	O(1)
08	<code>reorder_list</code>	O(n)	O(1)
09	<code>intersection_of_two_linked_lists</code>	O(n+m)	O(1)
10	<code>lru_cache</code>	O(1) per op	O(capacity)

Every problem here is O(1) extra space except the LRU cache, whose O(capacity) space *is* the point (it's a cache) - not a missed optimization.

Common interview follow-ups to be ready for

- `reverse_linked_list`: "can you do it recursively?" — yes, but recursive reversal uses $O(n)$ call-stack space, so the iterative version here is the space-optimal answer; know both.
- `linked_list_cycle_ii`: "prove why the second phase's meeting point is the cycle entrance" — interviewers frequently push on the math here specifically, not just whether the code works.
- `palindrome_linked_list` / `reorder_list`: "what if you can't mutate the input?" — both currently mutate the second half via reversal; the workaround is copying values into an array first (trades the $O(1)$ space guarantee for $O(n)$).
- `remove_nth_node_from_end`: "what if n is invalid (larger than the list length)?" — current code doesn't guard this; be ready to discuss what should happen (raise, no-op, or the problem guarantees valid n , as LeetCode does here).
- `intersection_of_two_linked_lists`: "what if the lists don't intersect?" — both pointers hit `None` at the same step and the loop condition `a is not b` becomes `None is not None` -> `False`, so it correctly returns `None` without a special case.
- `lru_cache`: "how would you make this thread-safe?" — a lock around `get/put` is the standard answer; also be ready for "what would an LFU (least *frequently* used) cache need instead?" (a second layer of buckets keyed by frequency).

Self-check before moving on

You should be able to, without looking at the code:

- Write the 3-line reversal core from memory.
- Explain why a dummy head removes the need for a "first node" special case.
- Walk through both phases of Floyd's cycle-start algorithm on a small hand-drawn example.
- Explain why `lru_cache` needs a *doubly* (not singly) linked list.
- Derive `reorder_list` on paper by naming the three sub-steps it's built from.

If any of those aren't fluent yet, this topic isn't "done" per the README's definition — revisit before moving to `05_stacks_queues`.

Programs

01. Reverse Linked List

01_reverse_linked_list.py

Explanation

Problem: Given the head of a singly linked list, reverse it and return the new head.

Source : LeetCode 206 - Reverse Linked List

Example:

Input: 1 -> 2 -> 3 -> 4 -> 5

Output: 5 -> 4 -> 3 -> 2 -> 1

Idea: walk the list once, and at each node flip its `.next` pointer to point backward instead of forward. `prev` tracks the already-reversed portion; `next_node` must be saved *before* overwriting `curr.next`, or the rest of the list becomes unreachable.

Code

```
from list_node import to_linked_list, to_python_list

def reverse_list(head):
    prev = None
    curr = head

    while curr:
        next_node = curr.next
        curr.next = prev
        prev = curr
        curr = next_node

    return prev

if __name__ == "__main__":
    tests = [
        ([1, 2, 3, 4, 5], [5, 4, 3, 2, 1]),
        ([], []),
        ([1], [1]),
        ([1, 2], [2, 1]),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        head = to_linked_list(inp)
        got = to_python_list(reverse_list(head))
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

02. Merge Two Sorted Lists

02_merge_two_sorted_lists.py

Explanation

Problem: Given the heads of two sorted linked lists, merge them into one sorted list by splicing together the existing nodes (no new nodes), and return the head.

Source : LeetCode 21 - Merge Two Sorted Lists

Example:

Input: l1 = 1 -> 2 -> 4, l2 = 1 -> 3 -> 4

Output: 1 -> 1 -> 2 -> 3 -> 4 -> 4

Idea: a dummy head sidesteps the "what's the first node" special case

- start **tail** at the dummy, always attach the smaller of the two current nodes to **tail.next**, advance both **tail** and whichever list contributed. Once one list runs out, the other is already sorted and can just be attached wholesale.

Code

```
from list_node import ListNode, to_linked_list, to_python_list

def merge_two_lists(l1, l2):
    dummy = ListNode()
    tail = dummy

    while l1 and l2:
        if l1.val <= l2.val:
            tail.next = l1
            l1 = l1.next
        else:
            tail.next = l2
            l2 = l2.next
        tail = tail.next

    tail.next = l1 if l1 else l2
    return dummy.next

if __name__ == "__main__":
    tests = [
        ([1, 2, 4], [1, 3, 4], [1, 1, 2, 3, 4, 4]),
        ([], [], []),
        ([], [0], [0]),
        ([2], [1], [1, 2]),
    ]

    for i, (a, b, expected) in enumerate(tests, 1):
        got = to_python_list(merge_two_lists(to_linked_list(a), to_linked_list(b)))
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

03. Linked List Cycle

03_linked_list_cycle.py

Explanation

Problem: Given the head of a linked list, determine if it has a cycle (some node's `.next` eventually points back to an earlier node), using $O(1)$ extra space.

Source : LeetCode 141 - Linked List Cycle

Example:

Input: 3 -> 2 -> 0 -> -4 -> (back to node with value 2)

Output: True

Idea: Floyd's cycle detection (fast-slow pointers), first previewed in 03_two_pointers_sliding_window/10_find_the_duplicate_number on a plain array. Same idea here on real nodes: if a cycle exists, the fast pointer (2 steps/iteration) will eventually lap the slow pointer (1 step/iteration) inside the loop and they'll land on the same node. If the list is acyclic, `fast` simply reaches the end (None) first.

Code

```
from list_node import ListNode

def has_cycle(head):
    slow = fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True

    return False

def _build_cyclic(values, pos):
    """pos = index the tail connects back to, or -1 for no cycle."""
    if not values:
        return None
    nodes = [ListNode(v) for v in values]
    for i in range(len(nodes) - 1):
        nodes[i].next = nodes[i + 1]
    if pos != -1:
        nodes[-1].next = nodes[pos]
    return nodes[0]

if __name__ == "__main__":
    tests = [
        ([3, 2, 0, -4], 1, True),
        ([1, 2], 0, True),
        ([1], -1, False),
        ([], -1, False),
        ([1, 2, 3], -1, False),
    ]

    for i, (values, pos, expected) in enumerate(tests, 1):
        head = _build_cyclic(values, pos)
        got = has_cycle(head)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

04. Linked List Cycle II

04_linked_list_cycle_ii.py

Explanation

Problem: Given the head of a linked list, return the node where the cycle begins, or None if there is no cycle. O(1) extra space.

Source : LeetCode 142 - Linked List Cycle II

Example:

Input: 3 -> 2 -> 0 -> -4 -> (back to node with value 2)

Output: the node with value 2

Idea: two-phase Floyd's algorithm. Phase 1 (same as 03) finds a meeting point somewhere inside the cycle - not necessarily its start. Phase 2 uses the algorithm's key property: resetting one pointer to **head** and advancing both pointers one step at a time from there, they meet again exactly at the cycle's entrance. (Proof sketch: if the entrance is **k** steps from head, the meeting point in phase 1 is also **k** steps before the entrance when measured going around the cycle - the math works out so both walks of length **k** land in the same place.)

Code

```
from list_node import ListNode

def detect_cycle(head):
    slow = fast = head
    met = False

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            met = True
            break

    if not met:
        return None

    ptr = head
    while ptr is not slow:
        ptr = ptr.next
        slow = slow.next

    return ptr

def _build_cyclic(values, pos):
    if not values:
        return None, None
    nodes = [ListNode(v) for v in values]
    for i in range(len(nodes) - 1):
        nodes[i].next = nodes[i + 1]
    entry = None
    if pos != -1:
        nodes[-1].next = nodes[pos]
```

```

        entry = nodes[pos]
    return nodes[0], entry

if __name__ == "__main__":
    tests = [
        ([3, 2, 0, -4], 1),
        ([1, 2], 0),
        ([1], -1),
        ([], -1),
        ([1, 2, 3], -1),
    ]

    for i, (values, pos) in enumerate(tests, 1):
        head, expected_entry = _build_cyclic(values, pos)
        got = detect_cycle(head)
        status = "PASS" if got is expected_entry else "FAIL"
        got_val = got.val if got else None
        expected_val = expected_entry.val if expected_entry else None
        print(f"Test {i}: {status} (got {got_val}, expected {expected_val})")

```

05. Middle Of Linked List

05_middle_of_linked_list.py

Explanation

Problem: Given the head of a singly linked list, return the middle node. If there are two middle nodes (even length), return the second one.

Source : LeetCode 876 - Middle of the Linked List

Example:

Input: 1 -> 2 -> 3 -> 4 -> 5

Output: node with value 3

Input: 1 -> 2 -> 3 -> 4 -> 5 -> 6

Output: node with value 4 (second of the two middles)

Idea: fast-slow pointers again, but this time the *destination* itself (where slow ends up when fast runs out) is the answer, not a meeting point. *fast* moves twice as fast as *slow*, so by the time *fast* reaches the end, *slow* has covered exactly half the distance - landing on the middle in a single pass, no need to count the length first.

Code

```

from list_node import to_linked_list

def middle_node(head):
    slow = fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

```

```

        return slow

if __name__ == "__main__":
    tests = [
        ([1, 2, 3, 4, 5], 3),
        ([1, 2, 3, 4, 5, 6], 4),
        ([1], 1),
        ([1, 2], 2),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got_node = middle_node(to_linked_list(inp))
        got = got_node.val if got_node else None
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

06. Remove Nth Node From End

06_remove_nth_node_from_end.py

Explanation

Problem: Given the head of a linked list, remove the nth node from the end of the list and return the head. One pass.

Source : LeetCode 19 - Remove Nth Node From End of List

Example:

Input: 1 -> 2 -> 3 -> 4 -> 5, n = 2

Output: 1 -> 2 -> 3 -> 5

Idea: two pointers held **n** nodes apart. Advance **fast** n steps first to create the gap, then advance both together until **fast** falls off the end - at that point **slow** sits exactly at the node *before* the one that needs removing. A dummy node before **head** handles the edge case of removing the head itself uniformly, so no separate "is this the first node" branch is needed.

Code

```

from list_node import ListNode, to_linked_list, to_python_list

def remove_nth_from_end(head, n):
    dummy = ListNode(0, head)
    fast = slow = dummy

    for _ in range(n):
        fast = fast.next

    while fast.next:
        fast = fast.next
        slow = slow.next

    slow.next = slow.next.next
    return dummy.next

if __name__ == "__main__":

```

```

tests = [
    ([1, 2, 3, 4, 5], 2, [1, 2, 3, 5]),
    ([1], 1, []),
    ([1, 2], 1, [1]),
    ([1, 2], 2, [2]),
]

for i, (inp, n, expected) in enumerate(tests, 1):
    got = to_python_list(remove_nth_from_end(to_linked_list(inp), n))
    status = "PASS" if got == expected else "FAIL"
    print(f"Test {i}: {status} (got {got}, expected {expected})")

```

07. Palindrome Linked List

07_palindrome_linked_list.py

Explanation

Problem: Given the head of a singly linked list, return True if it reads the same forwards and backwards. O(1) extra space (no copying into an array).

Source : LeetCode 234 - Palindrome Linked List

Example:

Input: 1 -> 2 -> 2 -> 1

Output: True

Idea: combines three techniques from this folder in sequence:

1. Find the middle (05's fast-slow trick).
2. Reverse the second half in place (01's reversal trick).
3. Walk the first half and the reversed second half together, comparing values - a palindrome check is just an equality check between a list and its own reverse.

This avoids O(n) extra space (no array copy) at the cost of temporarily mutating the list's second half - acceptable here since nothing else needs the original structure afterward.

Code

```

from list_node import to_linked_list

def is_palindrome(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    prev = None
    curr = slow
    while curr:
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt

    left, right = head, prev
    while right:

```

```

        if left.val != right.val:
            return False
        left = left.next
        right = right.next

    return True

if __name__ == "__main__":
    tests = [
        ([1, 2, 2, 1], True),
        ([1, 2], False),
        ([1], True),
        ([1, 2, 3, 2, 1], True),
        ([], True),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = is_palindrome(to_linked_list(inp))
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

08. Reorder List

08_reorder_list.py

Explanation

Problem: Given the head of a singly linked list $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow$

$L_{n-1} \rightarrow L_n$, reorder it in place to:

$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$

Source : LeetCode 143 - Reorder List

Example:

Input: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

Output: $1 \rightarrow 4 \rightarrow 2 \rightarrow 3$

Idea: three steps, each one reused from earlier in this folder:

1. Find the middle (05) and split the list into two halves.
2. Reverse the second half (01).
3. Merge the two halves by alternating nodes from each - similar shape to 02's merge, but here it's a strict alternation instead of a comparison-driven merge, since the two halves aren't sorted relative to each other.

Modifies the list in place; returns nothing (matches LeetCode's "do not return anything" signature convention for in-place problems).

Code

```

from list_node import to_linked_list, to_python_list

def reorder_list(head):
    if not head or not head.next:
        return

    slow = fast = head
    while fast and fast.next:

```

```

        slow = slow.next
        fast = fast.next.next

    second = slow.next
    slow.next = None
    prev = None
    curr = second
    while curr:
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt
    second = prev

    first = head
    while second:
        tmp1, tmp2 = first.next, second.next
        first.next = second
        second.next = tmp1
        first = tmp1
        second = tmp2

if __name__ == "__main__":
    tests = [
        ([1, 2, 3, 4], [1, 4, 2, 3]),
        ([1, 2, 3, 4, 5], [1, 5, 2, 4, 3]),
        ([1], [1]),
        ([1, 2], [1, 2]),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        head = to_linked_list(inp)
        reorder_list(head)
        got = to_python_list(head)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

09. Intersection Of Two Linked Lists

09_intersection_of_two_linked_lists.py

Explanation

Problem: Given the heads of two singly linked lists, return the node at which they intersect (share the exact same node onward), or None if they don't intersect. O(1) extra space.

Source : LeetCode 160 - Intersection of Two Linked Lists

Example:

listA-only: 4 -> 1 -\

6 -> 8 -> 4 -> 5 (shared tail)

listB-only: 5 -> 6 -/

Output: the node with value 6 (start of the shared tail)

Idea: if listA has length a and listB has length b, and they share a tail of length c, then walking A-then-B covers (a + b) total nodes, and so does walking B-then-A. Both walks end up traversing the exact

same total distance, so if you switch a pointer to the *other* list's head the instant it hits the end of its own list, both pointers arrive at the intersection point (or both hit None simultaneously, if there is none) at the same step - no need to compute lengths or alignment offsets up front.

Code

```
from list_node import ListNode

def get_intersection_node(head_a, head_b):
    if not head_a or not head_b:
        return None

    a, b = head_a, head_b
    while a is not b:
        a = a.next if a else head_b
        b = b.next if b else head_a

    return a

def _build_intersection(list_a_only, list_b_only, common):
    common_head = None
    common_tail = None
    for v in common:
        node = ListNode(v)
        if common_head is None:
            common_head = node
        else:
            common_tail.next = node
            common_tail = node

    def _prepend(values, tail):
        dummy = ListNode()
        curr = dummy
        for v in values:
            curr.next = ListNode(v)
            curr = curr.next
        curr.next = tail
        return dummy.next

    return _prepend(list_a_only, common_head), _prepend(list_b_only, common_head), common_head

if __name__ == "__main__":
    tests = [
        ([4, 1], [5, 6, 1], [8, 4, 5]),
        ([2, 6, 4], [1, 5], []),
        ([1], [], [])
    ]

    for i, (a_only, b_only, common) in enumerate(tests, 1):
        head_a, head_b, expected_node = _build_intersection(a_only, b_only, common)
        got = get_intersection_node(head_a, head_b)
        status = "PASS" if got is expected_node else "FAIL"
        got_val = got.val if got else None
        expected_val = expected_node.val if expected_node else None
        print(f"Test {i}: {status} (got {got_val}, expected {expected_val})")
```

10. Lru Cache

10_lru_cache.py

Explanation

Problem: Design a Least Recently Used (LRU) cache with fixed capacity, supporting `get(key)` and `put(key, value)` in $O(1)$ time each. `get` returns -1 if the key isn't present. `put` evicts the least recently used entry when inserting would exceed capacity. Both `get` and `put` count as "using" a key (move it to most-recently-used).

Source : LeetCode 146 - LRU Cache

Example:

```
cache = LRUCache(2)
cache.put(1, 1)
cache.put(2, 2)
cache.get(1) -> 1 (1 is now most recently used)
cache.put(3, 3) (evicts 2, the least recently used)
cache.get(2) -> -1 (evicted)
```

Idea: this is the one problem in the folder needing a *doubly* linked list, not singly. A dict alone gives $O(1)$ key lookup but no notion of recency order; a linked list alone gives $O(1)$ reordering but no $O(1)$ key lookup. Combine them: the dict maps key -> node, and a doubly linked list keeps nodes ordered by recency (most-recent right after a sentinel head, least-recent right before a sentinel tail). Doubly linked is required specifically because removing a node needs to patch up both its neighbor's pointers in $O(1)$ - a singly linked list would need to walk from the head to find a node's predecessor first, which breaks the $O(1)$ guarantee.

Code

```
class _DListNode:
    def __init__(self, key=0, val=0):
        self.key = key
        self.val = val
        self.prev = None
        self.next = None

class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.cache = {} # key -> _DListNode
        self.head = _DListNode() # sentinel, head.next = most recently used
        self.tail = _DListNode() # sentinel, tail.prev = least recently used
        self.head.next = self.tail
        self.tail.prev = self.head

    def _remove(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def _add_to_front(self, node):
        node.next = self.head.next
```

```

        node.prev = self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1
        node = self.cache[key]
        self._remove(node)
        self._add_to_front(node)
        return node.val

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            self._remove(self.cache[key])

        node = _DListNode(key, value)
        self.cache[key] = node
        self._add_to_front(node)

        if len(self.cache) > self.capacity:
            lru = self.tail.prev
            self._remove(lru)
            del self.cache[lru.key]

if __name__ == "__main__":
    cache = LRUCache(2)
    ops = [
        (cache.put, (1, 1), None),
        (cache.put, (2, 2), None),
        (cache.get, (1,), 1),
        (cache.put, (3, 3), None),    # evicts key 2
        (cache.get, (2,), -1),
        (cache.put, (4, 4), None),    # evicts key 1
        (cache.get, (1,), -1),
        (cache.get, (3,), 3),
        (cache.get, (4,), 4),
    ]

    for i, (fn, args, expected) in enumerate(ops, 1):
        got = fn(*args)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} ({fn.__name__}{args} -> got {got}, expected {expected})")

```